

AI-Powered Blockchain Smart Voting System with Real-Time Intelligent Fraud Detection

A comprehensive architectural and engineering manual explaining protocols, security design, algorithms, pinpoint workflows, pros, cons, and operations.

System Target:	Cryptographically Secure, Tamper-Evident Digital Voting Demonstrator
Core Pillars:	Solidity Smart Contracts, Next.js 14 App Router, Dexie.js PWA Queue, FastAPI ML Microservice
Consensus Environment:	EVM-compatible Hardhat Local / Sepolia Testnet Node (Chain ID: 31337 / 11155111)
AI Security Models:	Rule-Engine Velocity Scrutiny + Unsupervised Isolation Forest Anomaly Detection
Scope Designation:	Academic Prototype & Research Demonstrator (Flag-only, no automatic voter disenfranchisement)
Generated:	September 2026

EXECUTIVE ABSTRACT & PROBLEM FORMULATION

Traditional electronic voting systems suffer from centralized vulnerability vectors, single-point database corruptions, and opaque recount audits. Conversely, purely decentralized public voting struggles with high gas fees, voter identity privacy, biometric verification, and detection of syndicated vote-rigging. This project bridges this divide by synthesizing an **EVM-based cryptographic ledger** with an **Explainable Machine Learning Anomaly Detection Microservice** and an **Offline-First IndexedDB Progressive Web Application**. This document breaks down every layer of the system: why each dependency was selected, pinpoint implementation algorithms, exact workflows, operational strengths, real-world constraints, and running instructions.

1. High-Level Architecture & Component Breakdown

The system is organized into a modular decoupled architecture: the **Client & Orchestration Layer (Web)**, the **Immutable Ledger Layer (Smart Contracts)**, and the **Intelligent Heuristic Layer (AI Microservice)**. Each layer enforces strict separation of concerns, ensuring that failure or latency in one layer does not compromise ballot integrity.

Layer	Technology Stack	Key Responsibilities & Pinpoint Functions
Frontend & PWA Client	Next.js 14 (App Router), TypeScript, TailwindCSS, Dexie.js (IndexedDB), face-api.js, WebSockets (Socket.IO)	• Responsive voter ballot UX & admin monitoring dashboard. • Client-side face landmark liveness check (eye aspect ratio & head yaw). • Offline vote queueing via IndexedDB when network drops. • Auto-reconnect & idempotent background sync via Service Worker. • Live WebSocket push updates for fraud telemetry & tallies.
API & Relayer Gateway	Next.js Server Actions & Route Handlers, Ethers.js v6, MongoDB / In-Memory Fallback Store	• Gasless voter experience: Relayer signs transactions to Hardhat/Sepolia node using pre-funded admin keys. • Salted voter identity commitment hashing: keccak256(voterId + salt). • Off-chain metadata persistence (IP telemetry, review annotations). • Resilient database adapter switching seamlessly to in-memory mode if MongoDB is absent.
Blockchain Ledger Layer	Solidity (v0.8.24), Hardhat, OpenZeppelin Contracts (ReentrancyGuard, Ownable)	• VoterRegistry.sol: Stores pseudonymized commitments; prevents unauthorized voter eligibility tampering. • Election.sol: Strict lifecycle state machine (Created, Open, Paused, Closed) and candidate registry. • Ballot.sol: Checks-effects-interactions pattern, non-reentrant double-voting prevention, and transparent tally indexing.
AI Fraud Microservice	FastAPI, Python 3.13, Scikit-Learn (Isolation Forest), NumPy, Pandas, Pydantic v2	• Real-time REST endpoint (/predict) analyzing voting velocity, IP clustering, and submission intervals. • Unsupervised anomaly scoring via Isolation Forest. • Benford's Law distribution analysis for post-election audit. • Non-blocking flag generator (flags suspicious records for human review without voter disenfranchisement).

2. Rationales & Comparative Justifications (Why It Is Used)

1. Solidity Smart Contracts & EVM Architecture:

Traditional centralized SQL databases permit root administrators or malicious database operators to alter tally tables, backdate timestamps, or purge audit logs without detection. By enforcing voting rules at the EVM bytecode level, once a ballot transaction is mined with consensus, no actor—not even the system creator or election supervisor—can delete or reassign that vote. Every ballot emits a cryptographically sealed `VoteCast(uint256 indexed electionId, bytes32 indexed voterCommitment, uint256 candidateId, uint256 timestamp)` event that forms a perpetual, independently verifiable receipt tree.

2. Next.js 14 App Router & Server Actions (Gasless Relayer):

Requiring everyday citizens to install browser crypto wallets (e.g. MetaMask), acquire native gas tokens (ETH), and manage hexadecimal private keys introduces an insurmountable friction barrier. The architecture implements a **Trusted Relayer Pattern**: voters verify their credentials through WebAuthn/MFA, and the Next.js backend uses an authorized deployer wallet to sign and broadcast the EVM transaction on their behalf. This achieves zero-gas, one-click voter participation while retaining non-repudiable blockchain immutability.

3. Offline-First PWA with Dexie.js (IndexedDB) & Client Idempotency:

Real-world elections often occur in rural areas or network-congested polling centers where cellular connectivity fluctuates. Instead of failing or stranding votes, Dexie.js provides a structured browser storage queue. When connectivity drops, votes are signed client-side with a unique UUID `clientRequestId` and staged in IndexedDB. When the network is restored, the Service Worker background sync flushes the queue. The server and smart contract verify `clientRequestId` uniqueness, guaranteeing **exactly-once delivery**.

4. FastAPI & Scikit-Learn Microservice:

Node.js is single-threaded and ill-suited for heavy numerical matrix evaluations and statistical density modeling. Python's scientific ecosystem (FastAPI, NumPy, Pandas, Scikit-Learn) allows sub-50ms inference times. FastAPI provides native asynchronous I/O and Pydantic request schema validation, making it an ideal lightweight microservice to pair with the Next.js runtime.

3. Complete End-to-End Operational Lifecycle

The steps below articulate the exact path of a voter from credential authentication through fraud screening to on-chain confirmation:

Step 1: Multi-Factor Authentication

Voter logs in using their national ID / email and OTP. To satisfy biometric presence without violating biometrics storage laws, face-api.js processes webcam frames purely in browser memory, verifying eye-blink ratio and head rotation to confirm biological liveness. Raw biometric images are never saved to disk or network.

Step 2: Cryptographic Commitment Generation

To decouple the citizen's real-world identity from their ballot, the client/server hashes the voter ID with a secret cryptographic salt: `voterCommitment = keccak256(voterId + salt)`. The voter's name and PII are strictly excluded from the blockchain ledger.

Step 3: Ballot Cast & Client Queueing

The voter selects their candidate. The payload { `electionId`, `candidateId`, `voterCommitment`, `clientRequestId`, `clientTelemetry` } is prepared. If offline, Dexie.js stores it in IndexedDB. If online, it is immediately dispatched via POST to `/api/vote`.

Step 4: AI Fraud Telemetry Inspection

Before on-chain broadcast, the Next.js server relays submission metadata (timestamp, IP address, device user-agent hash, submission interval) to the FastAPI AI service at `http://127.0.0.1:8001/predict`. The service evaluates velocity thresholds and Isolation Forest anomaly boundaries, returning a fraud risk score [0.0 - 1.0] and diagnostic tags.

Step 5: Relayer Execution & Smart Contract Validation

The server's ethers.js relayer wallet invokes `Ballot.castVote(electionId, candidateId, voterCommitment)`. The smart contract checks: (a) election is Open; (b) voter commitment has registered eligibility; (c) `hasVoted[electionId][voterCommitment] == false`; (d) candidate exists. If all pass, `hasVoted` is set to true and tally is incremented.

Step 6: Real-Time Telemetry & WebSocket Push

The transaction hash, block number, and AI fraud score are persisted in the audit database. Simultaneously, the server emits a WebSocket event (`new-vote`) to the live Admin Dashboard, instantly rendering updated candidate vote bars, turnout meters, and fraud flag notifications.

Pinpoint Implementation: Ballot.sol Checks-Effects-Interactions Guard

```
// contracts/contracts/Ballot.sol
function castVote(
    uint256 electionId,
    uint256 candidateId,
    bytes32 voterCommitment
) external nonReentrant returns (bool) {
    // 1. CHECKS
    ElectionInfo storage election = elections[electionId];
    if (election.status != ElectionStatus.Open) revert ElectionNotOpen();
    if (block.timestamp < election.startTime || block.timestamp > election.endTime) revert OutsideElectionHours();
    ;
    if (!voterRegistry.isEligible(voterCommitment)) revert VoterNotRegistered();
    if (hasVoted[electionId][voterCommitment]) revert AlreadyVoted();
    if (candidateId == 0 || candidateId > election.candidateCount) revert InvalidCandidate();

    // 2. EFFECTS (State updated BEFORE any external interaction)
    hasVoted[electionId][voterCommitment] = true;
    voteCounts[electionId][candidateId] += 1;
    election.totalVotesCast += 1;

    // 3. INTERACTIONS (Event emission)
    emit VoteCast(electionId, voterCommitment, candidateId, block.timestamp);
    return true;
}
```

4. AI Fraud Detection Layer: Algorithms & Mathematical Basis

A critical innovation of this project is the integration of multi-layered fraud detection that monitors anomalies without relying on opaque, unexplainable black-box models. The engine operates on two complementary tiers: **Deterministic Rule-Based Velocity Filtering** and **Unsupervised Isolation Forest Scoring**.

Detection Mechanism	Mathematical / Logical Rule	Identified Attack Vector
IP Subnet Velocity Throttling	Votes from single IP > MAX_VOTES_PER_IP (5) within sliding window T_WINDOW (60s).	Syndicate proxy stuffing, automated bot farm hijacking, or co-located rogue network hubs.
Rapid Submission Burst	Time-to-submit $\Delta t < 1200\text{ms}$ between page render and biometric submission.	Script-driven programmatic headless browser submission bypassing human UI friction.
Duplicate Salt / Commitment	Occurrence of keccak256 commitment in in-flight cache > 1 within short epoch.	Replay attacks attempting to submit duplicate ballots before on-chain confirmation.
Isolation Forest Anomaly Scoring	Tree partition depth anomaly score: $s(x, n) = 2^{(-E(h(x)) / c(n))}$ where $s > 0.65$ indicates anomaly.	Subtle multi-dimensional coordinated vote clustering across jitter, device headers, and timestamps.
Benford's Law First-Digit Analysis	$P(d) = \log_{10}(1 + 1/d)$ compared against candidate precinct vote tallies via Chi-Square test (χ^2).	Post-election algorithmic ballot stuffing or artificial vote fabrication across electoral precincts.

CRITICAL ETHICAL GUARDRAIL: HUMAN-IN-THE-LOOP AUDIT PRINCIPLE

Under international democratic election standards, **no algorithmic model may unilaterally disenfranchise a voter or reject an otherwise valid ballot at the point of submission.** In this system, when the AI flags a ballot (risk score > 0.65), the transaction is *still submitted to the blockchain* to preserve the citizen's franchise. However, the transaction hash is tagged with isFlagged: true and queued on the Admin Audit Dashboard for bipartisan human scrutiny and post-election forensic review.

5. System Advantages & Architectural Strengths (Pros)

- **Cryptographic Tamper-Proofing:** Once a block is sealed on the Ethereum-compatible ledger, mathematical immutability guarantees that no malicious administrative privilege or database compromise can rewrite historical votes or inflate tallies.
- **Zero Gas Voter Accessibility:** The backend transaction relayer subsidizes gas costs transparently. Voters never need cryptocurrency, wallet extensions, or blockchain domain knowledge to cast legitimate ballots.
- **Resilient Offline Queueing:** Leveraging Dexie.js (IndexedDB) and PWA Service Workers, remote polling locations can continue collecting verified votes even during temporary fiber cuts or power outages, synchronizing reliably when connectivity returns.
- **Zero PII on Public Ledgers:** Voters are identified only by irreversible salted hashes (keccak256(voterId + salt)). Government-issued identity numbers, names, and biometric templates are strictly excluded from the blockchain blocks.
- **Explainable AI Telemetry:** Rather than using an untraceable deep neural network, the fraud engine outputs concrete human-readable explanation tags (e.g. 'IP_BURST_SUSPICIOUS', 'RAPID_SUBMISSION_INTERVAL') empowering election officials to make justifiable decisions.
- **Sub-Second Transparent Tallies:** Election tallies update in real time as blocks mine, eliminating days or weeks of manual ballot counting while retaining an on-chain verifiable audit trail for every ballot.

6. Risks, Real-World Limitations & Trade-offs (Cons)

For an honest academic defense, it is paramount to document the system's limitations, vulnerabilities, and real-world implementation constraints:

Constraint / Risk Area	Nature of Vulnerability	Mitigation Strategy / Future Research
Absence of Coercion Resistance	Online remote voting cannot prevent vote-buying or an abusive family member standing over the voter's shoulder demanding a specific ballot selection.	Implementation of receipt-free cryptographic voting systems such as MACI (Minimal Anti-Collusion Infrastructure) allowing voters to re-cast ballots secretly.
Relayer Centralization Vector	The centralized Next.js relayer wallet holds the private key to submit transactions. A compromised server could theoretically censor specific transactions before they reach the chain.	Transitioning to decentralized threshold relayer networks (e.g. Chainlink Automation, Gelato, or Biconomy Account Abstraction ERC-4337).
Biometric Client-Side Spoofing	Client-side face-api.js runs inside the user's browser runtime. An attacker with custom browser extensions or virtual webcams could feed pre-recorded video frames.	Hardware-backed secure enclaves (Apple FaceID / WebAuthn FIDO2 keys) providing cryptographically signed hardware attestation tokens.
Public Chain Throughput Limits	Public Ethereum L1 processes ~15-30 TPS, far too slow for national elections with tens of millions of simultaneous voters during peak voting hours.	Deploying on Zero-Knowledge Rollups (zk-SNARKs / Starknet) or dedicated private Avalanche/Polygon supernets capable of 5,000+ TPS.
AI Synthetic Training Drift	The current Isolation Forest model is trained on synthetic demographic distributions. It may produce false positives on real voting patterns (e.g. voters using communal satellite Wi-Fi).	Calibrating hyper-parameters on empirical historical election turnout logs and continuous active-learning updates.

7. Operational Setup & Verification Guide

To launch the entire distributed architecture on a clean system, follow these sequential steps in three dedicated terminal consoles:

Terminal 1: EVM Blockchain Local Network

```
cd contracts
npm install
npx hardhat node
# Starts local JSON-RPC consensus engine on 127.0.0.1:8545 with 20 pre-funded test accounts (10,000 ETH each).
```

Terminal 2: AI Fraud Detection Microservice

```
cd ai-service
.\venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --host 127.0.0.1 --port 8001 --reload
# Launches FastAPI server with OpenAPI docs at http://127.0.0.1:8001/docs.
```

Terminal 3: Next.js 14 Web Application & Relayer

```
cd web
npm install
npm run dev
# Compiles Next.js App Router on http://localhost:3000.
# In-memory database fallback activates automatically if MongoDB is not running locally.
```

8. Summary & Academic Conclusion

This project validates the technical feasibility of synthesizing decentralized consensus ledgers with intelligent statistical inference. By enforcing deterministic ballot uniqueness at the smart contract layer, offloading identity hashing to secure client/server pipelines, and utilizing machine learning exclusively for non-destructive fraud telemetry, the system achieves a balanced synergy of **integrity**, **privacy**, and **operational resilience**.